

Elevator Simulator

You are tasked with developing an elevator system that has **two elevators** and ten floors. Your program should include a user interface that allows users to input the desired floor number, and the current floor for each elevator should be displayed.

Key focus — the two elevators must operate *simultaneously*. While one elevator is moving, the other can independently receive a request and move **at the same time**. The elevators must not block each other (it should **not** be “one finishes, then the other starts”). How you achieve this concurrency — threads, async, an event loop, processes, etc. — is up to you.

Requirements

1. Define a class called `Elevator` that has the following attributes and methods:

Attributes:

- `current_floor`: an integer representing the current floor of the elevator.

2. Methods:

- `display_floor()`: a method that displays the current floor of the elevator.
- `move(current: int, floor: int)`: a method that moves the elevator to the specified floor. This method should update the `current_floor` attribute and call `display_floor()` to display the current floor.

3. Create two `Elevator` objects named `elevator1` and `elevator2`.
4. Implement a user interface that allows users to input their current floor and the desired floor number. The program should move the elevators to the specified floors and display their current floors.
5. **Concurrent operation (required)**. Both elevators must be able to move **at the same time**. While `elevator1` is travelling, the user can still issue a command to `elevator2`, and it should start moving **immediately and independently**. The display should keep updating both elevators' current floors in real time, so that two in-progress journeys are visibly happening together.

Optional (Bonus)

1. The elevator moving speed is one floor per second. (Combined with requirement 5, this makes the simultaneous movement observable — e.g. both elevators counting through their floors at the same time.)
2. Implement Unit Tests.
3. **Remote Control**: Implement a client and server into this simulator. This is the use case for a security guard monitoring the elevators remotely from the central control room.
4. **Multiple elevators with realistic dispatching**. Extend the simulator from two elevators to **N elevators** serving the same building. When a user presses a button on a floor (a hall call), the **system** should automatically decide **which** elevator answers — for example the nearest available car, taking each car's current direction into account — instead of the user choosing a specific elevator. The goal is to minimise passenger waiting time, the way a real elevator bank works.

Submission

As completed, please put your code in your GitHub repository and let us know the link.



*Notes: The key is to simulate how two elevators work **simultaneously**. Therefore, the user interface definition of this simulator can be re-defined if you have an idea to better demonstrate it.*